

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2



Manual de Usuario - Codex Latinus

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Agosto de 2026.

Índice

Descripción.....	3
1. Instalación y Requisitos del Sistema.....	3
2. Descripción de la Interfaz y Menú Lateral.....	4
2.1 Editor de Texto.....	4
2.2 Nuevo.....	5
2.3 Abrir.....	6
2.4 Guardar.....	6
2.5 AST.....	7
2.6 Tabla de Símbolos.....	7
2.7 Tabla de Errores Léxicos.....	8
2.8 Tabla de Errores Sintácticos.....	8
2.9 Tabla de Errores Semánticos.....	9
2.10 Pila de Procesos.....	9
3. Estructura del Código y Descripción del Lenguaje en Codex Latinus.....	10
A. Tipos de Datos y Jerarquía (Validación Implícita).....	10
B. Expresiones y Operadores.....	10
C. Declaraciones y Asignaciones.....	10
D. Uso de Estructuras.....	11
E. Estructuras de Control y Ciclos.....	11
F. Funciones y Bloque Principal (MAIOR>).....	12
4. Compilación y Consola de Resultados.....	13
Consola de Resultados.....	13
Consola de Traducción.....	13

Manual de Usuario: Codex Latinus IDE

Descripción

Bienvenido al manual oficial del entorno de desarrollo **Codex Latinus**, una aplicación de escritorio diseñada para la creación, análisis y ejecución de programas bajo la sintaxis del lenguaje.

1. Instalación y Requisitos del Sistema

Al tratarse de una aplicación de escritorio, se debe de cumplir con los siguientes requisitos previos antes de iniciarla:

- **Requisito de Software:** Tener instalado **Java 22** (JDK 22) en tu sistema operativo. Puedes verificarlo abriendo tu terminal o consola y ejecutando el comando:

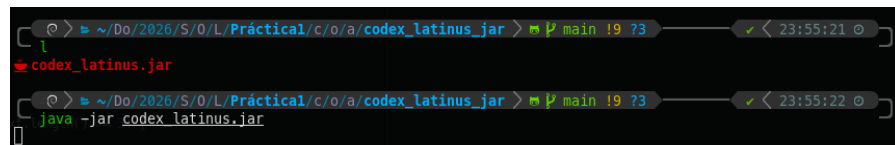
`java -version`

```
java -version
java version "22.0.2" 2024-07-16
Java(TM) SE Runtime Environment (build 22.0.2+9-70)
Java HotSpot(TM) 64-Bit Server VM (build 22.0.2+9-70, mixed mode, sharing)
```

(Debe mostrar una versión compatible con Java 22).

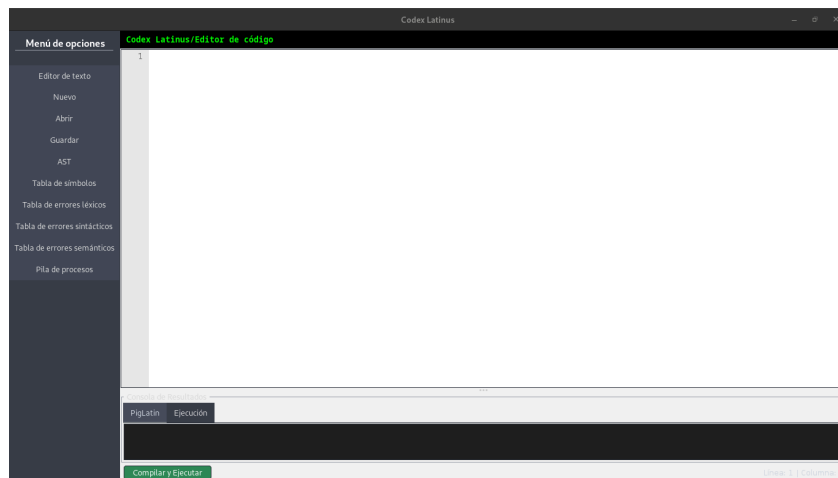
- **Instalación y Ejecución:**
 1. Descarga el paquete de distribución de la aplicación de escritorio o el archivo ejecutable empaquetado (el archivo .jar ejecutable).
 2. Si ejecutas el archivo .jar directamente desde tu terminal, utiliza el siguiente comando:

`java -jar codex_latinus.jar`



```
~/Do/2026/S/O/L/Práctical/c/o/a/codex_latinus.jar > main !9 73
codex_latinus.jar
~/Do/2026/S/O/L/Práctical/c/o/a/codex_latinus.jar > main !9 73
java -jar codex_latinus.jar
```

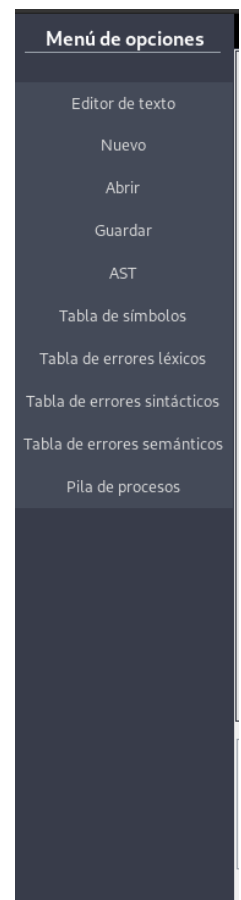
La interfaz gráfica de la aplicación de escritorio se abrirá automáticamente en tu pantalla.



2. Descripción de la Interfaz y Menú Lateral

El panel izquierdo del entorno organiza las herramientas de navegación y depuración del sistema:

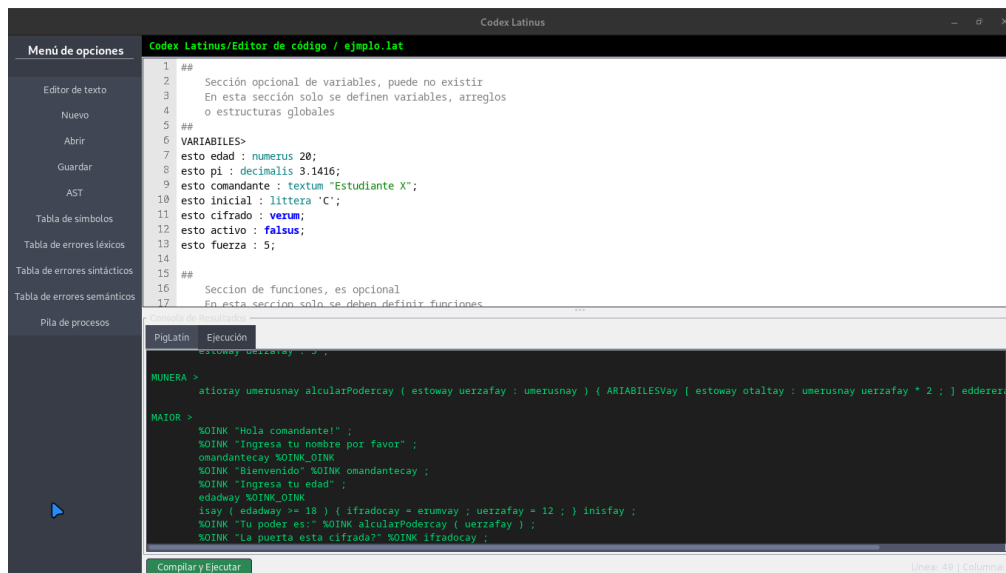
- **Editor de texto:** Permite visualizar y modificar el código fuente actual (por ejemplo, ejemplo.lat).
- **Nuevo:** Permite crear un archivo nuevo con extensión .lat.
- **Abrir:** Permite abrir y seguir editando algún archivo con extensión .lat.
- **Guardar:** Permite guardar el archivo de texto editable .lat, además de guardar el archivo .pig que tendrá el la traducción a PigLatin, ambos se guardarán en una carpeta seleccionada por usted.
- **AST:** Permite visualizar el Árbol de Análisis Sintáctico generado por el compilador.
- **Tabla de Símbolos:** Acá podrá visualizar la tabla de símbolos generada por el compilador.
- **Tabla de errores léxicos:** Acá podrá ver la tabla de errores léxicos si existieran, si no hay verá un mensaje.
- **Tabla de errores sintácticos:** Acá podrá ver la tabla de errores sintácticos si existieran, si no hay verá un mensaje.
- **Tabla de errores semánticos:** Acá podrá ver la tabla de errores semánticos si existieran, si no hay verá un mensaje.
- **Pila de Procesos:** Acá podrá visualizar la pila de procesos generada por el compilador.



2.1 Editor de Texto

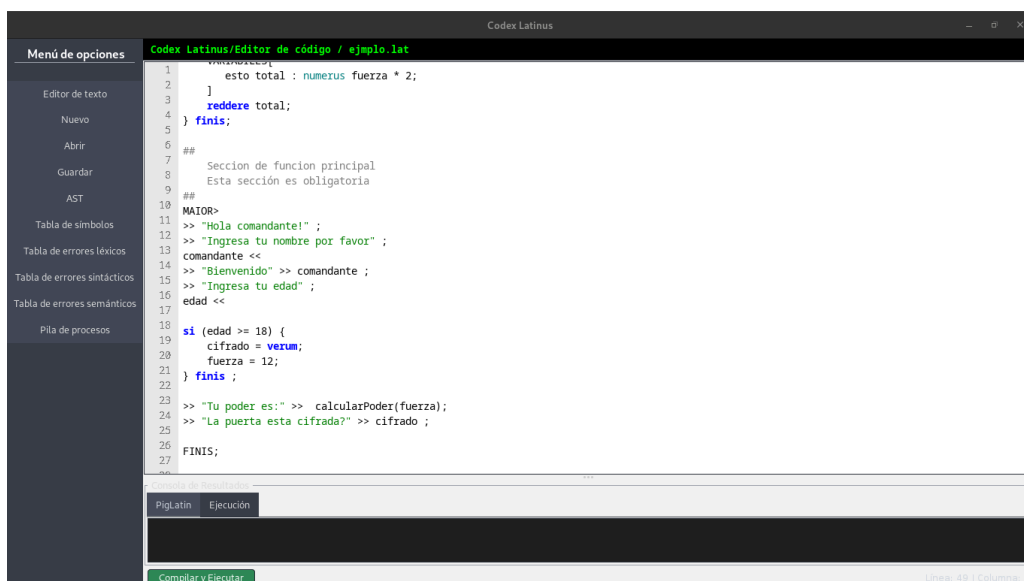
En este apartado podrá ingresar texto en formato de Codex Latinus, posteriormente en la parte inferior podrá visualizar las diferentes salidas, a PigLatin y Traducción, también el proceso de Compilar y

Ejecutar que le permitirá traducir el código escrito. El texto estará formateado con colores para poder distinguir entre variables, palabras reservadas, tipos de datos, funciones, salidas de consola.



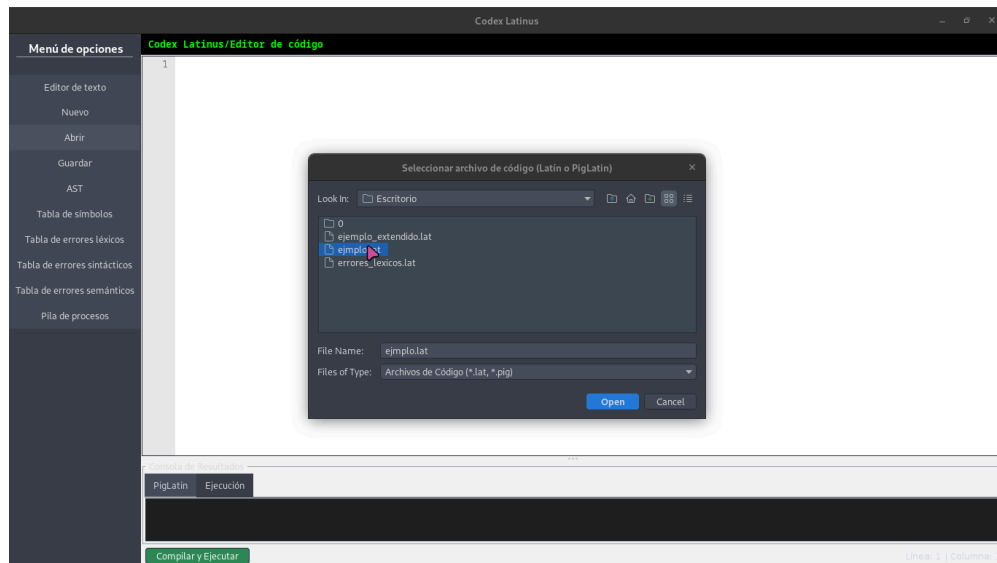
2.2 Nuevo

Acá podrá crear un nuevo archivo en blanco, que posteriormente podrá guardarlo, al guardarlo se creará el archivo nombre_archivo.lat.



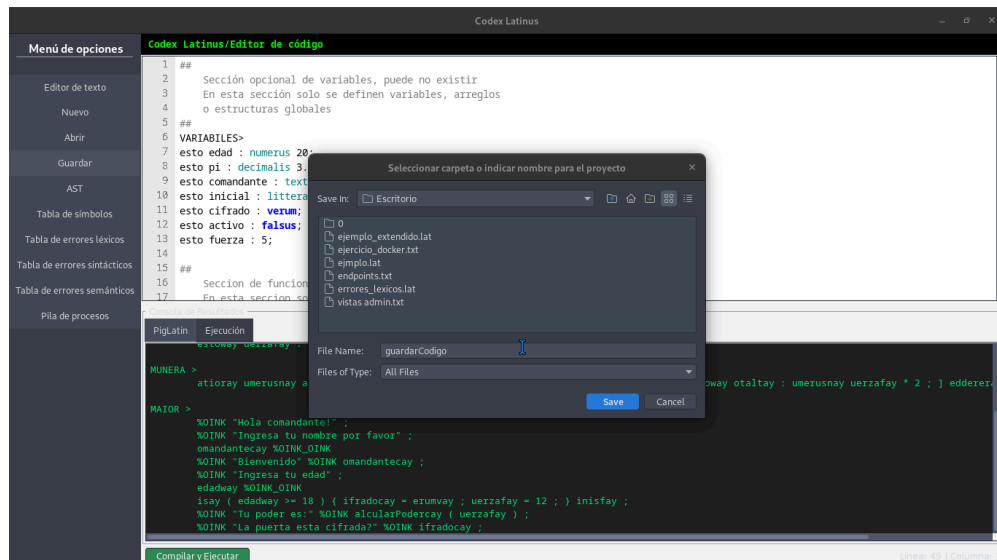
2.3 Abrir

Podrá abrir un archivo con extensión .lat, el cual contendrá código Codex Latinus que podrá editar si desea para obtener otros resultados.



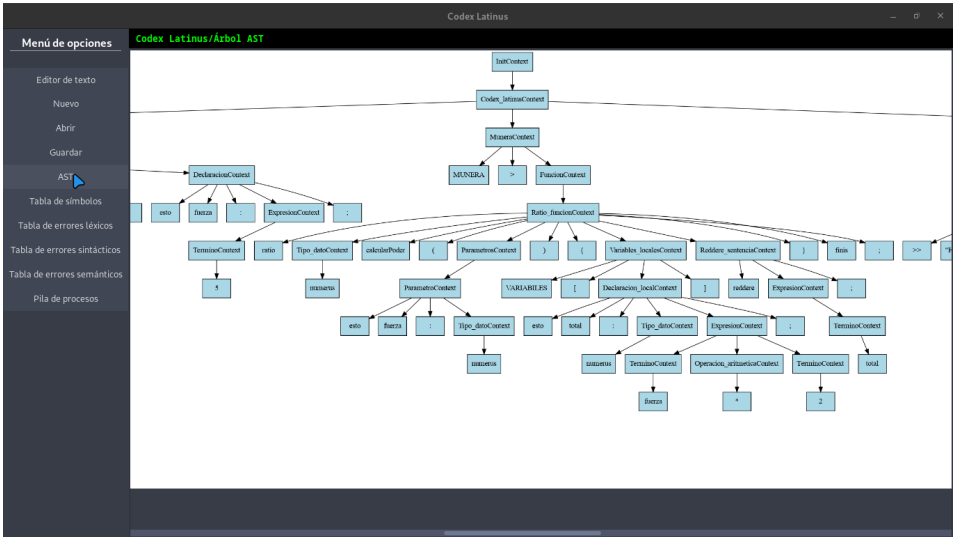
2.4 Guardar

Este botón le permitirá guardar el archivo .lat que contendrá su código, además también contendrá el archivo .pig, que contiene el código traducido a PigLatin.



2.5 AST

Este botón le permitirá visualizar el Árbol de Análisis Sintáctico, este árbol es generado por el compilador y le permitirá visualizar cómo se procesa el código en cada nodo.



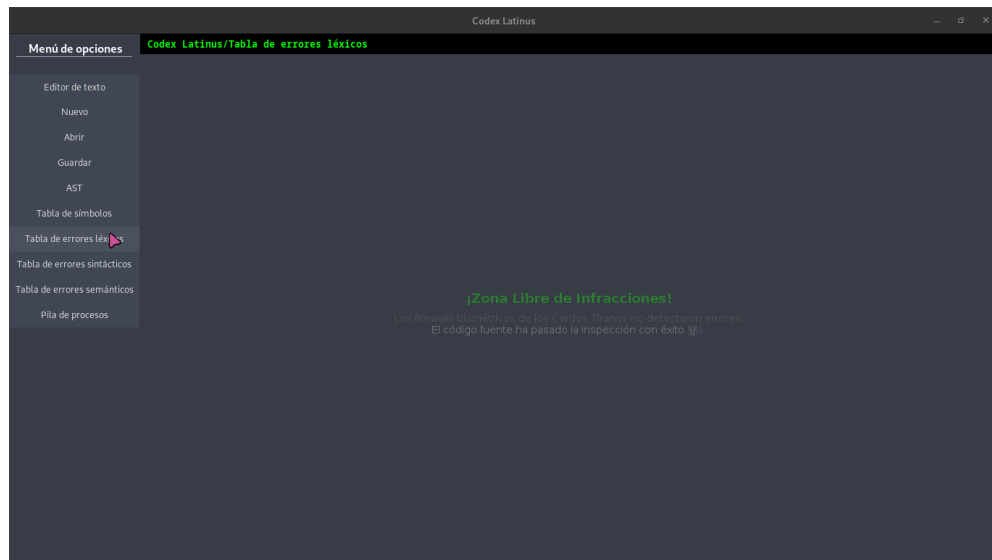
2.6 Tabla de Símbolos

En este apartado podrá visualizar la tabla de símbolos generada por el compilador, en él podrá visualizar cada variable, función que fue declarada y modificada.

Codex Latinus									
Menú de opciones	Codex Latinus/Tabla de símbolos								
	Nombre	Tipo de Dato	Valor	Tipo	Rol	Ámbito	Línea	Columna	
Editor de texto	cifrado	boolean	true	Primitivo	variable	global	11	5	
Nuevo	calcularPoder	numerus	null	Primitivo	ratio	global	21	14	
Abrir	comandante	textum	Estudiante X	Primitivo	variable	global	9	5	
Guardar	pi	decimalis	3.1416	Primitivo	variable	global	8	5	
AST	inicial	littera	C	Primitivo	variable	global	10	5	
	edad	numerus	20	Primitivo	variable	global	7	5	
	fuerza	numerus	12	Primitivo	variable	global	13	5	
	activo	boolean	false	Primitivo	variable	global	12	5	

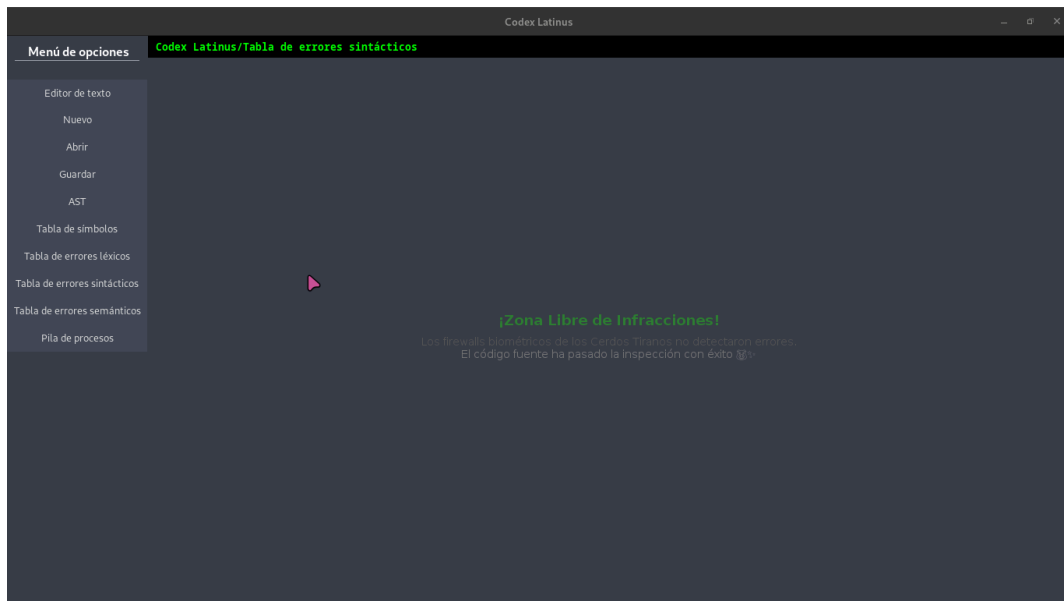
2.7 Tabla de Errores Léxicos

Acá podrá visualizar los tipos de errores léxicos ocurridos al compilar, de no encontrar errores verá un mensaje indicándolo.



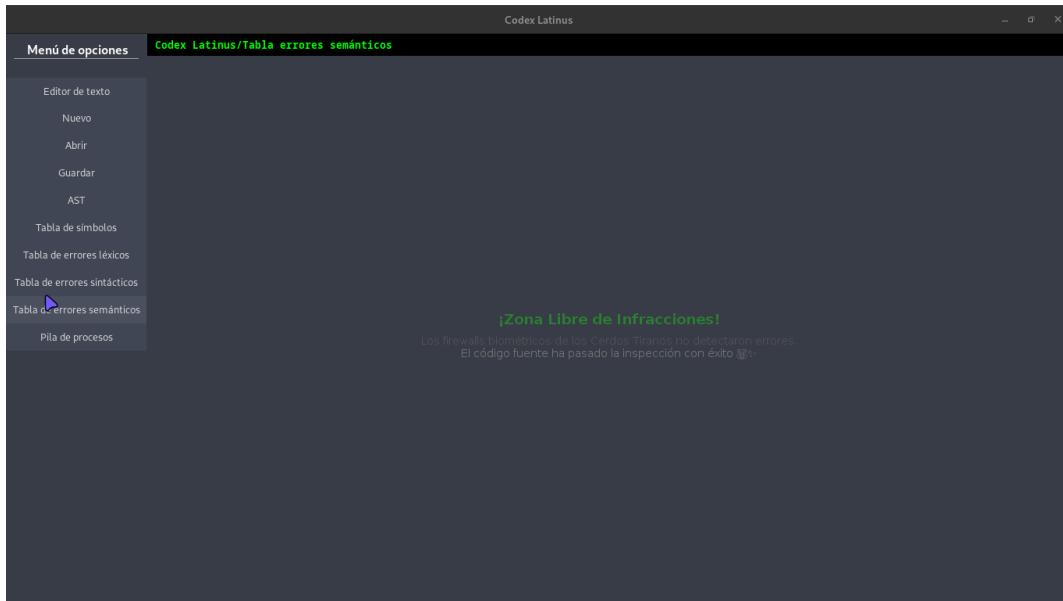
2.8 Tabla de Errores Sintácticos

Acá podrá visualizar los tipos de errores sintácticos ocurridos al compilar, de no encontrar errores verá un mensaje indicándolo.



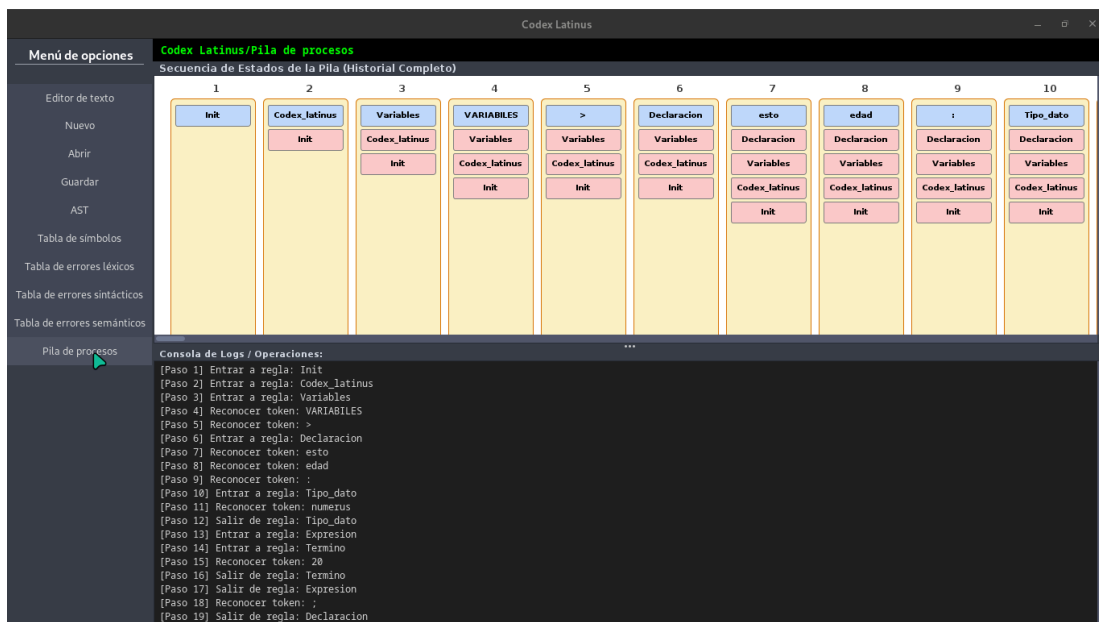
2.9 Tabla de Errores Semánticos

Acá podrá visualizar los tipos de errores semánticos ocurridos al compilar, de no encontrar errores verá un mensaje indicándolo.



2.10 Pila de Procesos

En este apartado podrá ver la pila de proceso que ocurrió durante la compilación, podrá verlo de manera visual y en la parte inferior podrá ver un log con los pasos ocurridos.



3. Estructura del Código y Descripción del Lenguaje en Codex Latinus

El lenguaje de entrada está escrito en latín y combina definiciones modulares, estructuras de datos, control de flujo y un bloque de inicio obligatorio.

A. Tipos de Datos y Jerarquía (Validación Implícita)

El compilador implementa un análisis semántico estricto para la verificación de tipos (Type Checking) mediante inferencia implícita. Las operaciones consideran la jerarquía de tipos de mayor a menor valor:

1. **textum** (Textos / Cadenas) - El único tipo incompatible con operaciones matemáticas, permitiendo únicamente concatenación mediante la suma.
2. **decimalis** (Números decimales)
3. **numerus** (Números enteros)
4. **littera** (Caracteres individuales)
5. **verum / falsus** (Valores lógicos)

B. Expresiones y Operadores

Las expresiones permiten operar valores primitivos (los arreglos y estructuras no son primitivos, pero los datos que albergan sí). Los operadores permitidos son:

- **Aritméticos:** +, -, *, / (Estándar universal).
- **Relacionales:** ==, !=, <, >, >=, <= (Estándar universal).
- **Lógicos:** &&, || (And y Or).
- **Negación:** non (Negación universal).
- **Suma/Resta abreviada:** ++, -- (Suma o resta una unidad).

C. Declaraciones y Asignaciones

Todas las declaraciones de variables deben ir dentro de una sección especial del código.

- **Variables simples:**

esto mi_entero : numerus 10;

*esto total : numerus fuerza * 2;*

esto nombre : textum "Somos la resistencia";

esto gravedad : decimalis 9.81;

esto inicial : littera 'a';

- **Booleanos:** Para un orden absoluto (verum) o caos (falsus), o bien utilizando la sintaxis rápida:

esto mi_booleano : verum;

esto otro_booleano : falsus;

- **Arreglos:** Permiten almacenar colecciones de cualquier tipo primario o estructura. Las posiciones inician en 0:

series mis_enteros[2] : numerus {1, 1};

series nombres[2] : textum {"Hola", "Adios"};

nombres[0] = "Capitán Espárragos";

D. Uso de Estructuras

Permiten definir tipos de datos personalizados validando que no se repitan los nombres de los atributos:

```
structura Persona {  
    esto nombre: textum;  
    esto edad: numerus;  
} finis;
```

```
esto mi_personaje : Persona {  
    nombre: "UserName",  
    edad: 28  
};
```

E. Estructuras de Control y Ciclos

- **Condicionales (si):** Requieren una condición estrictamente booleana; de lo contrario, se marca un error de "Corrupción de Flujo".

```

si (x > 10 && y < 5) {

    // Instrucciones

} aliter {

    // Instrucciones alternativas

} finis;

```

- **Ciclos:**
 - *dum* (condición) { ... } *finis*; (Ejecución iterativa mientras se cumpla).
 - *facere* { ... } *dum* (condición); (Ejecución de al menos una vez).
 - *per* (esto *i* : *numerus* 0; *i* < 10; *i*++) { ... } (Ciclo con iterador).
 - Incluyen las palabras reservadas *perge* (continuar) e *interrumpe* (romper) para el control de flujo interno.

F. Funciones y Bloque Principal (MAIOR>)

- **Funciones sin retorno (actio) y con retorno (ratio):** Las variables locales solo se definen al principio de la función dentro del bloque VARIABILES. Las funciones *ratio* utilizan la sentencia *reddere* para devolver un valor validando que todos los caminos alcancen un retorno correcto.

```

ratio numerus calcularPoder(esto fuerza : numerus) {

    VARIABILES [

        esto total : numerus fuerza * 2;

    ]

    reddere total;

} finis;

```

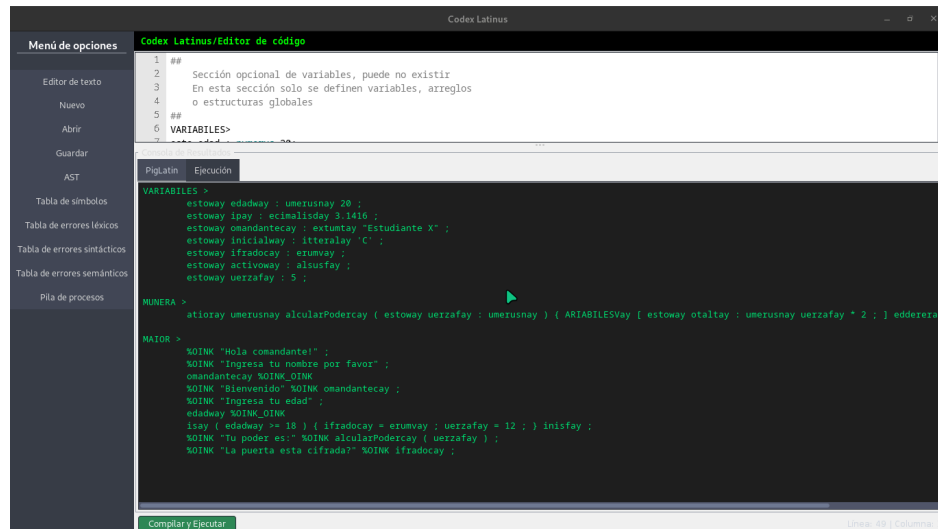
- **Bloque Principal (MAIOR>):** Sección obligatoria donde inicia la ejecución lógica del programa.
- **Funciones Especiales de E/S:**
 - Lectura: << (lee texto sin almacenar) o *mi_textum* << (lee y guarda en variable).
 - Impresión: >> "Holaaaaa"; o concatenadas >> *mi_string* >> *mi_textum*;

4. Compilación y Consola de Resultados

La parte inferior de la interfaz está dedicada al control de la ejecución del programa:

Consola de Resultados

En esta verá el código transformado en PigLatin



Consola de Traducción

Acá podrá ver el código transcrito a código humano.

